

45-Day AI Engineering Study Plan

Ed Donners + AI Engineering from Scratch + AI-Engineering Roadmap

Interleaved 1.5-month full-time curriculum
8.5 hours/day, AI-assisted code generation

Resource	Nominal Time	Your Time	Why Faster
Ed Donners (33h video + 8 projects)	8 weeks part-time	~5 days	1.5x playback + follow along
AI-Eng Roadmap (12 projects)	5 months full-time	~22 days	AI generates boilerplate; you learn architecture
From-scratch [CORE] lessons	~47 hours	~6 days	Read + code key lessons only
Buffer / re-dos / interview prep	—	~12 days	Things will go wrong
TOTAL	—	~45 days	= 1.5 months at 8.5h/day

Prepared: August 2026 | Sagar Marthandan

Contents

1. The Strategy: Interleave, Don't Sequence
2. Phase 1: Foundations (Days 1-5)
3. Phase 2: RAG Deep Dive (Days 6-15)
4. Phase 3: Agents (Days 16-24)
5. Phase 4: Evaluation + Infrastructure (Days 25-34)
6. Phase 5: Capstone (Days 35-45)
7. 2-Month Extension (Days 46-60)
8. How to Use AI for Code Generation
9. What to Ruthlessly Skip

1. The Strategy: Interleave, Don't Sequence

The three resources overlap heavily. Ed Donners teaches the concept, from-scratch fills the depth, the roadmap proves the skill. Doing them in parallel per topic is 3x faster than doing each sequentially.

Ed Donners Week (concept)	From-Scratch [CORE] (depth)	Roadmap Project (proof)
~0.5 day	~0.5 day	~1.5 days

Legend

W1-W8 = Ed Donners Udemy course weeks (1.5x playback speed)

P#-L## = AI Engineering from Scratch Phase #, Lesson ##

P1-P12 = AI-Engineering Roadmap projects (portfolio pieces)

[CORE] = study alongside | **[DEEP]** = optional, if time permits | **[SKIP]** = you know this already

2. Phase 1: Foundations (Days 1-5)

Ed Donners Weeks 1-2. Core concepts: API calls, tokenization, transformers, tool calling, multimodal. First roadmap project setup.

Day	Ed Donners	From-Scratch	Roadmap
1	W1: Ollama, OpenAI API, tokens, frontier models (1.5x playback)	—	—
2	W1: Transformers overview, brochure generator (Project 1)	P7-L02, L03: Self-Attention + Multi-Head from scratch	—
3	W2: Gradio, streaming chatbot, system prompts	P7-L05: Full Transformer + P10-L01: Tokenizers	—
4	W2: Tool calling, airline assistant (Project 2), DALL-E/TTS	P10-L02: Build a Tokenizer from Scratch	—
5	W2: Multimodal, agentic AI intro	P10-L06, L07, L08: SFT, RLHF, DPO (skim)	P1: Setup repo, pgvector, embedding pipeline

3. Phase 2: RAG Deep Dive (Days 6-15)

Ed Donners Weeks 3-5. The most intensive phase. HuggingFace, model evaluation, and the full RAG stack: hybrid search, re-ranking, GraphRAG, evaluation. Roadmap Projects 1-3.

Day	Ed Donners	From-Scratch	Roadmap
6	W3: HuggingFace, pipelines, tokenizers	P7-L07: GPT Causal LM	P1: BM25 + vector search + hybrid
7	W3: Transformers low-level, inside LLaMA	P7-L12: KV Cache, Flash Attention	P1: Cross-encoder re-ranking, HyDE
8	W3: Running open-source models (Phi, Qwen, DeepSeek)	P10-L11: Quantization (skim)	P1: Finish + README + metrics → ■
9	W4: Model evaluation, code gen (Project 4)	P10-L10: Evaluation — Benchmarks	P2: Tenant isolation, pgvector RLS
10	W4: Reasoning models, business tasks, LLM competition	P5-L27: LLM Eval (RAGAS, DeepEval)	P2: RBAC, audit logs, citations
11	W5: RAG intro, vectors, vector DBs	P11-L06: RAG from scratch	P2: Finish + README → ■
12	W5: Embeddings, t-SNE, chunking, RAG pipeline	P5-L23: Chunking + P11-L07: Advanced RAG	P3: GraphRAG (Neo4j setup)
13	W5: RAG eval (MRR, NDCG, LLM-as-judge)	—	P3: Agentic RAG + decision framework
14	W5: Advanced RAG (query rewriting, re-ranking, GraphRAG)	—	P3: Multimodal RAG + finish → ■
15	W5: Knowledge worker (Project 5) + buffer	—	Buffer / re-do weak projects

4. Phase 3: Agents (Days 16-24)

Ed Donners Weeks 6-8. Traditional ML baselines (skim), fine-tuning GPT-4o-mini, QLoRA on LLaMA, and the multi-agent capstone. Roadmap Projects 4-6.

Day	Ed Donners	From-Scratch	Roadmap
16	W6: Datasets, traditional ML baselines, PyTorch NN	P3-L03: Backprop from Scratch (if not done)	P4: Setup, function calling, SQL generation
17	W6: Fine-tuning GPT-4o-mini (SFT), when FT fails	—	P4: Validation, execution, error recovery → ■
18	W6: Deep neural network (Project 6)	—	P5: Classification → extraction → validation
19	W7: LoRA, QLoRA quantization, hyperparameters	P11-L08: Fine-Tuning with LoRA & QLoRA	P5: Pydantic validation, routing → ■
20	W7: Dataset prep, base vs chat, training hyperparams	—	P6: LangGraph setup, orchestrator-worker
21	W7: W&B;, TRL SFT Trainer, monitoring, cross-entropy	—	P6: Critic loops, agent memory
22	W7: Full dataset training on A100, testing vs GPT-4o (Project 7)	—	P6: Finish + README → ■
23	W8: Agentic AI, Modal deployment, agent architectures	P14-L01, L02, L12: Agent Loop, Plan-Execute, Workflow Patterns	Buffer
24	W8: Multi-agent system (Project 8) + buffer	P14-L25, L28: Multi-Agent + Orchestration	Buffer / re-do

5. Phase 4: Evaluation + Infrastructure (Days 25-34)

Ed Donners is complete. Full focus on AI-Engineering Roadmap Projects 7-10: evaluation harness, prompt A/B testing, LLM serving gateway, PII guardrails. This is the tier that separates AI engineers from API callers.

Day	Ed Donners	From-Scratch	Roadmap
25	— (Ed Donners done)	—	P7: LLM-as-judge, faithfulness/relevance metrics
26	—	—	P7: Regression tests, context engineering → ■
27	—	—	P8: Prompt versioning, A/B test infrastructure
28	—	—	P8: Statistical significance, metric tracking → ■
29	—	—	P9: Semantic caching (Redis), model routing
30	—	—	P9: Cost tracking, rate limiting, failover → ■
31	—	—	P10: Presidio + spaCy NER, regex patterns
32	—	—	P10: LLM detection, GDPR logging → ■
33	—	—	Buffer / re-do weak projects
34	—	—	Buffer / polish READMEs / commit history

6. Phase 5: Capstone (Days 35-45)

Roadmap Projects 11-12. The portfolio centerpiece: a full customer support AI system combining RAG, agents, multi-agent orchestration, evaluation, guardrails, and monitoring. Then a technical case study communicating architecture decisions.

Day	Focus	Detail
35-36	P11: Architecture	RAG + agents + multi-agent + eval + guardrails + monitoring. Design the full system.
37-38	P11: Core pipeline	Ingestion → retrieval → agent response → eval loop
39-40	P11: Guardrails + deploy	PII redaction, cost tracking, Streamlit dashboard, Docker
41	P11: Finish → ■	README + metrics + architecture diagram
42-43	P12: Case study	Architecture docs, ADRs, metrics, failure analysis
44	P12: Finish → ■	Complete technical writeup
45	Final review	Polish all repos, update ROADMAP.md, push to GitHub

7. 2-Month Extension (Days 46-60)

If you have 2 months (60 days) instead of 1.5, use the extra 15 days to deepen understanding, rebuild without AI assistance, and prepare for interviews.

Days	Activity
46-48	Go back and do [DEEP] from-scratch lessons you skipped: P7-L14 Build Transformer, P10-L02 Build Tokenizer, P19-L69 E2E RAG
49-51	Rebuild your weakest roadmap project from scratch (no AI assistance) to prove you actually understand it
52-54	Write blog posts / LinkedIn articles about 2-3 projects (German market values written technical communication)
55-57	Interview prep: practice explaining each project's architecture, tradeoffs, and metrics out loud
58-60	Apply to jobs with the finished portfolio

8. How to Use AI for Code Generation (Without Cheating Yourself)

The trap: AI generates code, you don't understand it, interviewers destroy you. The fix is a disciplined workflow that ensures you own every line.

Step	Action	Why
1	Read the IMPLEMENTATION_PLAN.md for the project	Understand the architecture before any code is written
2	Ask AI to generate the code	AI handles boilerplate, imports, config
3	READ every line it produces. If you can't explain it, ask AI to explain it	This is where learning happens — in the reading, not the writing
4	Run it, break it intentionally, fix the break	Prove you understand the failure modes, not just the happy path
5	Modify 3 things without AI help: change the embedding model, swap the vector DB, add a new endpoint	If you can do this, you own the code. If you can't, go back to step 3

The Interview Test

Rule: for each project, you must be able to explain the architecture diagram, the data flow, and why you chose each technology — without looking at the code. That's what interviews test. If you can't draw the architecture on a whiteboard and explain why pgvector over Qdrant, why LangGraph over CrewAI, why cross-encoder re-ranking over bi-encoder — you don't own the project yet.

9. What to Ruthlessly Skip

In 1.5 months, every hour matters. Cut these without guilt — they either overlap with skills you already have or are depth you can revisit later.

Skip	Why
All from-scratch [SKIP] lessons	You know this — environment setup, basic Python, Git
All from-scratch [DEEP] lessons (in 1.5mo version)	No time; revisit in the 2-month buffer days 46-48
Ed Donners W6 traditional ML baselines (Random Forest, XGBoost)	You know this from data engineering / analytics
Ed Donners W6 Bag of Words / CountVectorizer	You know this — basic NLP text representation
Re-doing projects you already understand	Move forward — depth comes from new challenges, not repetition
Perfecting every project	8 solid projects beat 12 polished ones in 1.5 months. Ship, document, move on.

Resources

Ed Donners course: <https://www.udemy.com/course/llm-engineering-master-ai-and-large-language-models/>

AI Engineering from Scratch: <https://aiengineeringfromscratch.com>

GitHub repo: <https://github.com/rohitg00/ai-engineering-from-scratch>

AI-Engineering Roadmap: </home/sagar/Projects/AI-Engineering Roadmap/>

Topic mapping PDF: /home/sagar/Projects/Ed_Donners_vs_AI_Engineering_from_Scratch_Mapping.pdf

Interactive terminal learning: Run `npx skills add rohitg00/ai-engineering-from-scratch` then `/start - learning` in Claude Code, Cursor, or Codex for a placement quiz and personalized path.